Сетевое программирование

Современные программные приложения крайне редко существуют изолированно. Большинство из них взаимодействуют с другими программами, сервисами или устройствами через сеть: получают данные с серверов, отправляют запросы к веб-сервисам, обмениваются сообщениями в реальном времени.

В данной лекции мы перейдём от локальной работы с данными (файлы, память) к распределённому взаимодействию между приложениями, находящимися на разных компьютерах или в разных частях сети.

1. Основы сетевого взаимодействия

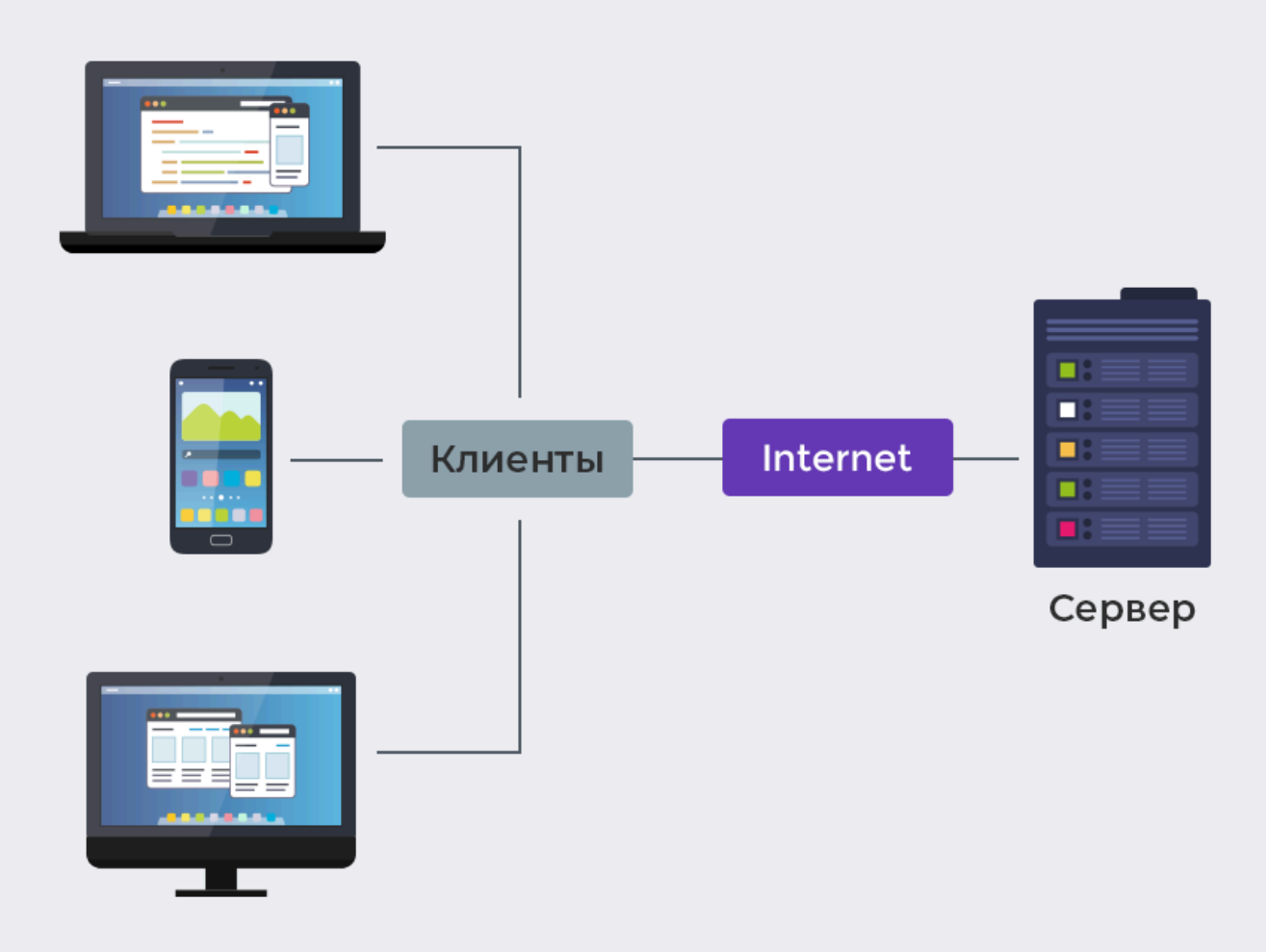
Прежде чем переходить к конкретным сетевым протоколам, необходимо разобраться в базовых понятиях, на которых строится любое сетевое взаимодействие. Понимание этих основ позволит осознанно работать с сетью, а не воспринимать её как «чёрный ящик».

Большинство сетевых приложений построены по модели **клиент–сервер**.

- **Клиент** — приложение, которое инициирует соединение и отправляет запросы.
- **Сервер** — приложение, которое ожидает подключения, принимает запросы и отправляет ответы.

Ключевые особенности модели:

- клиент всегда знает, к кому он обращается;
- сервер не знает клиентов заранее;
- один сервер может обслуживать множество клиентов одновременно.



Адресация в сети

Чтобы клиент мог подключиться к серверу, необходимо однозначно указать, **где он находится**. Для этого используются несколько уровней адресации.

IP-адрес — это уникальный числовой идентификатор узла в сети.
Основные версии:

- **IPv4** — 32-битный адрес (например, `192.168.1.10`);
- **IPv6** — 128-битный адрес (например `2001:0db8:85a3::8a2e:0370:7334`).

Порт — это логический идентификатор конкретного сетевого приложения на устройстве.

- диапазон портов: `0–65535`;
- один IP-адрес может обслуживать тысячи приложений через разные порты;
- сервер «слушает» определённый порт.

Например так мы подключимся к порту `8080` по адресу `127.0.0.1` — `127.0.0.1:8080`.

IP + порт однозначно определяет точку подключения.

Localhost (локальный хост) — это стандартное, зарезервированное имя (домен) для обозначения собственного компьютера, на котором в данный момент работает пользователь или программа. Оно указывает на то, что сетевой запрос не уходит в интернет, а обрабатывается внутри операционной системы, часто используя IP-адрес **127.0.0.1**.

DNS — доменные имена

Работать напрямую с IP-адресами неудобно, поэтому используется **DNS (Domain Name System)**.

DNS:

- сопоставляет доменные имена IP-адресам;
- позволяет использовать человекочитаемые адреса (например, `google.com` вместо IP-адреса).

С помощью этого кода, мы сможем получить список всех IP-адресов соответствующих доменному имени `google.com`.

```
IPHostEntry entry = Dns.GetHostEntry("google.com");
IPAddress[] addresses = entry.AddressList;
foreach(var address in addresses)
    Console.WriteLine(address);
```

Результат в консоли.

```
2a00:1450:4010:c0b::8a
2a00:1450:4010:c0b::64
2a00:1450:4010:c0b::65
2a00:1450:4010:c0b::8b
173.194.222.113
173.194.222.138
173.194.222.139
173.194.222.100
173.194.222.102
173.194.222.101
```

Соединение и обмен данными

Сетевое взаимодействие состоит из нескольких этапов:

1. Клиент инициирует подключение к серверу.
2. Устанавливается соединение.
3. Происходит обмен данными:
   - клиент отправляет данные;
   - сервер принимает и обрабатывает их;
   - сервер отправляет ответ.
4. Соединение закрывается или остаётся открытым.

Фактически данные передаются как поток байтов.

Передача данных и пакеты

На физическом уровне данные:

- разбиваются на небольшие части (пакеты);
- передаются независимо друг от друга;
- собираются обратно на стороне получателя.

Это означает, что:

- данные могут приходить не сразу;
- части сообщения могут быть получены по отдельности;
- приложение должно быть готово к частичному чтению данных.

Надёжность и задержки сети

Сеть по своей природе медленнее оперативной памяти и диска, а также подвержена задержкам и может быть нестабильной.

Это приводит к следующим последствиям:

- операции ввода-вывода могут выполняться долго;
- соединения могут разрываться;
- данные могут не дойти до адресата.

Поэтому сетевой код должен:

- быть асинхронным;
- корректно обрабатывать ошибки;
- учитывать таймауты и повторные попытки.

2. Протоколы передачи данных

Для того чтобы разные устройства и приложения могли корректно обмениваться данными по сети, они должны «говорить на одном языке». Эту роль выполняют **сетевые протоколы** — наборы правил, определяющих формат данных, порядок их передачи и способы обработки ошибок.

Сетевой протокол — это формальное соглашение, которое определяет:

- как устанавливается соединение;
- как передаются данные;
- в каком формате они отправляются;
- как обнаруживаются и обрабатываются ошибки.

Без протоколов:

- переданные байты были бы неинтерпретируемыми;
- невозможно было бы обеспечить совместимость между системами.

Многоуровневая модель сети

Сетевые протоколы организованы в виде уровней. Каждый уровень решает свою задачу и не зависит от внутренней реализации других уровней.

Чаще всего используется модель **TCP/IP**, состоящая из четырёх уровней:

1. **Прикладной уровень**
   - HTTP, FTP, SMTP, WebSocket и др.
2. **Транспортный уровень**
   - TCP, UDP.
3. **Сетевой уровень**
   - IP.
4. **Канальный уровень**
   - физическая передача данных.

Для программиста на C# наибольший интерес представляют прикладной и транспортный уровни.

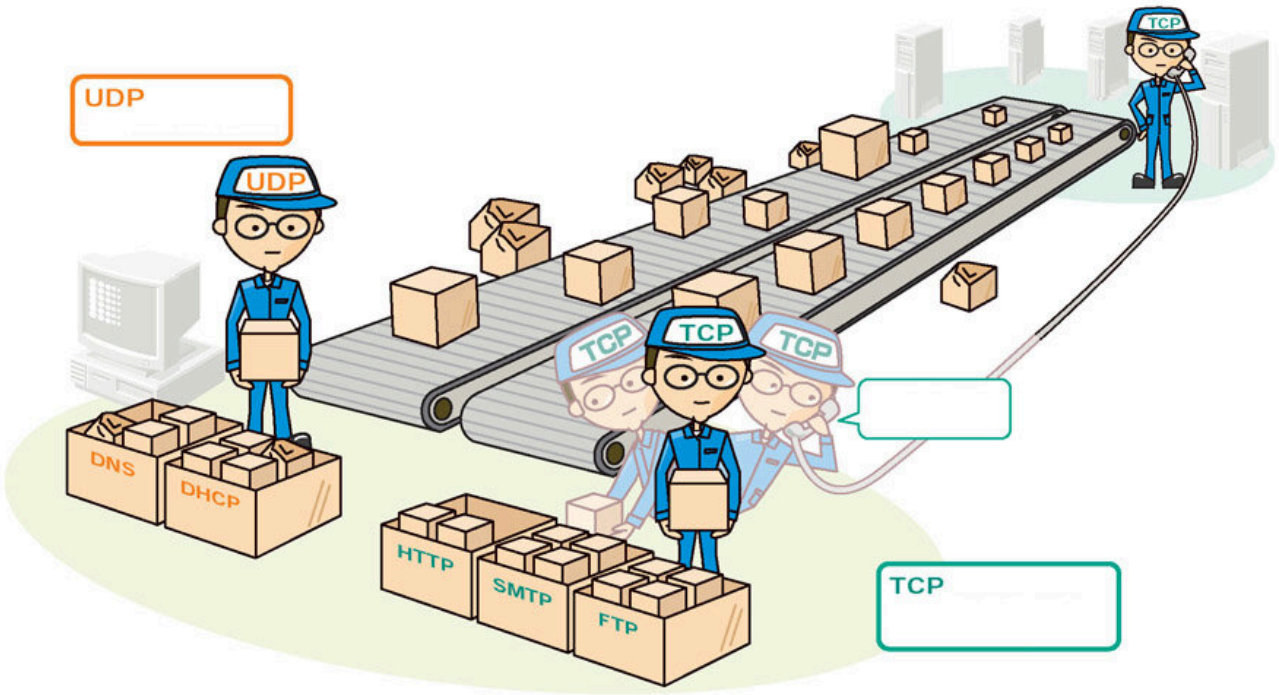
Транспортный уровень

Транспортный уровень отвечает за:

- доставку данных между приложениями;
- разбиение данных на пакеты;
- управление порядком передачи.

Основные протоколы транспортного уровня — **TCP** и **UDP**.

- **TCP (Transmission Control Protocol)** — этот протокол гарантирует доставку данных и их порядок, он устанавливает соединение между источником и получателем данных и адаптирует скорость передачи под сетевые условия. TCP используется тогда, когда важна целостность данных (веб-страницы, файловые загрузки).
- **UDP (User Datagram Protocol)** — этот протокол не устанавливает соединение, данные отправляются сразу, пакеты обрабатываются независимо (может быть потеря пакетов или нарушение порядка). Он используется тогда, когда важна скорость, а не надёжность (стриминг, онлайн-игры).



Сравнение TCP и UDP

Критерий	TCP	UDP
Соединение	Да	Нет
Гарантия доставки	Да	Нет
Порядок пакетов	Гарантирован	Не гарантирован
Скорость	Ниже	Выше

Прикладные протоколы

На основе TCP и UDP строятся протоколы прикладного уровня:

- **HTTP** — передача веб-запросов и ответов;
- **HTTPS** — HTTP с шифрованием;
- **FTP** — передача файлов;
- **SMTP** — отправка электронной почты;
- **WebSocket** — двустороннее взаимодействие в реальном времени.

Прикладные протоколы:

- определяют формат сообщений;
- добавляют семантику поверх передачи байтов;
- скрывают детали транспортного уровня.

3. URI и URL

При работе с сетью постоянно встречаются два похожих термина — **URI** и **URL**. Их часто путают или используют как синонимы, хотя между ними есть принципиальное различие.

URI (Uniform Resource Identifier) — это универсальный идентификатор ресурса. Он отвечает на вопрос «что это?» и однозначно идентифицирует ресурс, но не обязательно указывает, как до него добраться.

URL (Uniform Resource Locator) — это частный случай URI, который не только идентифицирует ресурс, но и указывает его местоположение и способ доступа. URL отвечает на вопрос «где это и как получить?».

На практике часто эти термины обозначают одно и то же — адрес ресурса в сети. Когда говорят «URI», имеют в виду путь вроде `/api/users/5`, а когда говорят «URL» — полный адрес вроде `https://api.example.com/api/users/5`.

Структура URL

Полный URL состоит из нескольких частей:

```
https://api.example.com:8080/users/42?sort=asc&page=1#section
  │         │              │     │         │              │
схема     домен           порт  путь    параметры       якорь
```

Разберём каждую часть:

Схема — протокол, по которому происходит обращение. Определяет способ передачи данных. Примеры: `http`, `https`, `ftp`, `ws`, `wss`.

Домен — адрес сервера. Может быть доменным именем (`api.example.com`) или IP-адресом (`192.168.1.10`). DNS преобразует доменное имя в IP-адрес перед подключением.

Порт — необязательная часть. Если не указан, используется порт по умолчанию для схемы: `80` для HTTP, `443` для HTTPS. Указывается через двоеточие после домена.

Путь — указывает на конкретный ресурс на сервере. Начинается с `/`, части разделяются слэшем. Путь отражает иерархию ресурсов: `/users/42/orders`.

Параметры запроса (query string) — начинаются с `?`, представляют собой пары `ключ=значение`, разделённые `&`. Используются для передачи дополнительных данных: `?sort=asc&page=1&limit=20`.

Якорь (fragment) — начинается с `#`, указывает на конкретную часть страницы. Якорь не отправляется на сервер, он обрабатывается только браузером на стороне клиента.

Работа с URI в C#: класс Uri

В .NET для работы с адресами используется класс `Uri` из пространства имён `System`. Он разбирает строку адреса на составные части и предоставляет удобный доступ к каждой из них:

```
var uri = new Uri("https://api.example.com:8080/users/42?sort=asc&page=1");

Console.WriteLine(uri.Scheme);       // https
Console.WriteLine(uri.Host);         // api.example.com
Console.WriteLine(uri.Port);         // 8080
Console.WriteLine(uri.AbsolutePath); // /users/42
Console.WriteLine(uri.Query);        // ?sort=asc&page=1
```

`Uri` также умеет строить адреса из частей — базового адреса и относительного пути:

```
var baseUri = new Uri("https://api.example.com/");
var relativeUri = new Uri("users/42", UriKind.Relative);
var fullUri = new Uri(baseUri, relativeUri);

Console.WriteLine(fullUri); // https://api.example.com/users/42
```

Построение адресов: класс UriBuilder

Если адрес нужно собирать динамически из отдельных частей, используется класс `UriBuilder`. Он позволяет задавать каждую часть URL по отдельности:

```
var builder = new UriBuilder();
builder.Scheme = "https";
builder.Host = "api.example.com";
builder.Path = "/users/42";

Console.WriteLine(builder.Uri); // https://api.example.com/users/42
```

Можно передать уже готовый адрес и изменить только нужные части:

```
var builder = new UriBuilder("https://api.example.com/users");
builder.Path = "/users/42";
builder.Port = 8080;

Console.WriteLine(builder.Uri); // https://api.example.com:8080/users/42
```

Работа с параметрами запроса

В параметры запроса нельзя вставлять строки напрямую: значения могут содержать спецсимволы или кириллицу, которые браузер и сервер не поймут. Их нужно закодировать с помощью `Uri.EscapeDataString`:

```
string city = "Нью-Йорк";
string encoded = Uri.EscapeDataString(city); // %D0%9D%D1%8C%D1%8E-
%D0%99%D0%BE%D1%80%D0%BA
```

Для удобной сборки строки запроса можно формировать строку через словарь:

```
var builder = new UriBuilder("https://api.example.com/users");
var queryParams = new Dictionary<string, string>
{
    { "sort", "asc" },
    { "page", "1" },
    { "search", "Иван Иванов" }
};
builder.Query = string.Join("&",
    queryParams.Select(kvp =>
        $"{Uri.EscapeDataString(kvp.Key)}={Uri.EscapeDataString(kvp.Value)}"
    )
);
Console.WriteLine(builder.Uri);
// https://api.example.com/users?
sort=asc&page=1&search=%D0%98%D0%B2%D0%B0%D0%BD%20%D0%98%D0%B2%D0%B0%D0%BD%D
0%BE%D0%B2
```

Экранирование особенно важно при работе с пользовательскими данными — без него специальные символы (`&`, `=`, пробелы, кириллица) сломают структуру URL или приведут к некорректному запросу.

4. Сокеты

Все высокоуровневые средства сетевого взаимодействия в .NET в конечном итоге построены поверх сокетов. Понимание сокетов позволяет глубже разобраться в том, как на самом деле работает сеть.

Сокет — это программный интерфейс для обмена данными между приложениями по сети.

С точки зрения приложения сокет:

- представляет конечную точку сетевого соединения;
- позволяет отправлять и получать байты;
- работает поверх транспортного протокола (TCP или UDP).

Фактически сокет связывает IP-адрес, порт и протокол передачи данных.

В .NET работа с сокетами осуществляется через пространство имён:

```
System.Net.Sockets
```

Основной класс: `Socket`

Он предоставляет полный контроль над сетевым взаимодействием, но требует:

- ручного управления соединением;
- обработки ошибок;
- аккуратной работы с ресурсами.

Типы сокетов

При создании сокета указываются три ключевых параметра:

1. **AddressFamily**
   - `InterNetwork` — IPv4;
   - `InterNetworkV6` — IPv6.
2. **SocketType**
   - `Stream` — потоковый сокет (TCP);
   - `Dgram` — датаграммный сокет (UDP).
3. **ProtocolType**
   - `Tcp`;
   - `Udp`.

Рассмотрим пример того, как создать TCP и UPD сокеты.

```
// TCP-сокет (потоковый)
Socket tcpSocket = new Socket(
    AddressFamily.InterNetwork, // IPv4
    SocketType.Stream,           // Потоковый тип (TCP)
    ProtocolType.Tcp
);

// UDP-сокет (датаграммный)
Socket udpSocket = new Socket(
    AddressFamily.InterNetwork,
    SocketType.Dgram,            // Датаграммный тип (UDP)
    ProtocolType.Udp
);
```

TCP-сервер на сокетах

Последовательность действий TCP-сервера:

1. Создание сокета.
2. Привязка к IP-адресу и порту (`Bind`).
3. Переход в режим ожидания подключений (`Listen`).
4. Принятие подключения (`Accept`).
5. Обмен данными.
6. Закрытие соединения.

```
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
IPEndPoint serverEndPoint = new IPEndPoint(serverIP, 8080);
tcpSocket.Bind(serverEndPoint); // Привязка к адресу и порту
// Начало прослушивания
tcpSocket.Listen(10); // Максимум 10 подключений в очереди
while (true)  {
    using Socket clientSocket = tcpSocket.Accept(); // Принятие подключений
    byte[] buffer = new byte[1024];// Обработка клиента
    int bytesReceived = clientSocket.Receive(buffer); // Чтение данных в
буфер
    string message = Encoding.UTF8.GetString(buffer, 0, bytesReceived); //
Переводим данные в строку
    Console.WriteLine($"Получено: {message}");
    string response = "Сообщение получено!";  // Ответ клиенту
    byte[] responseData = Encoding.UTF8.GetBytes(response); // Переводим
строку в массив байт и отправляем клиенту
    clientSocket.Send(responseData);
    clientSocket.Shutdown(SocketShutdown.Both); // Закрываем клиентское
подключение
}
```

TCP-клиент на сокетах

Клиентская сторона:

```
// Подключение к серверу
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
IPEndPoint serverEndPoint = new IPEndPoint(serverIP, 8080);
tcpSocket.Connect(serverEndPoint);

// Отправка данных
string message = "Привет, сервер!";
byte[] data = Encoding.UTF8.GetBytes(message);
tcpSocket.Send(data);

// Получение ответа
byte[] buffer = new byte[1024];
int bytesReceived = tcpSocket.Receive(buffer);
string response = Encoding.UTF8.GetString(buffer, 0, bytesReceived);
Console.WriteLine($"Ответ сервера: {response}");
```

Теперь после того как мы запустим сервер, а потом клиент то получим:

- На сервере `Получено: Привет, сервер!`
- На клиенте `Ответ сервера: Сообщение получено!`

UDP-сокеты

UDP-сокеты работают иначе:

- соединение не устанавливается;
- данные отправляются в виде отдельных сообщений.

Пример:

```
var udp = new Socket(
    AddressFamily.InterNetwork,
    SocketType.Dgram,
    ProtocolType.Udp
);

byte[] data = Encoding.UTF8.GetBytes("Сообщение");
udp.SendTo(data, new IPEndPoint(IPAddress.Loopback, 5001));
```

При использовании UDP:

- приложение само отвечает за логику доставки;
- возможна потеря сообщений.

5. TcpClient, TcpListener и UdpClient

Работа напрямую с сокетами даёт полный контроль над сетевым взаимодействием, но требует значительных усилий и приводит к громоздкому, трудно поддерживаемому коду. Поэтому в .NET предусмотрены **высокоуровневые сетевые классы**, которые инкапсулируют большинство низкоуровневых деталей и делают разработку сетевых приложений проще и безопаснее.

Классы `TcpClient`, `TcpListener` и `UdpClient` построены поверх `Socket`, скрывают детали работы с буферами и соединениями, интегрированы с потоками данных (`Stream`) и удобны для использования с `async/await`.

TcpListener — TCP-сервер

`TcpListener` предназначен для приёма входящих TCP-подключений. Типичный жизненный цикл сервера выглядит так: создание листенера, запуск прослушивания, ожидание клиентов в цикле, обработка каждого подключения, остановка сервера.

После принятия подключения `TcpListener` возвращает объект `TcpClient`, через который происходит дальнейший обмен данными. Удобство в том, что вместо ручной работы с байтовыми буферами можно сразу получить `NetworkStream` и обернуть его декораторами — `StreamReader` и `StreamWriter`.

```
var server = new TcpListener(IPAddress.Any, 8080);
server.Start();
Console.WriteLine("Сервер запущен, ожидание подключений...");

while (true)
{
    // Асинхронно ожидаем подключение клиента
    using TcpClient client = await server.AcceptTcpClientAsync();
    Console.WriteLine("Клиент подключился");

    await using NetworkStream ns = client.GetStream();
    using StreamReader reader = new StreamReader(ns);
    using StreamWriter writer = new StreamWriter(ns) { AutoFlush = true };

    // Читаем сообщение от клиента
    string? message = await reader.ReadLineAsync();
    Console.WriteLine($"Получено: {message}");

    // Отправляем ответ
    await writer.WriteLineAsync("Сообщение получено!");
}
```

Обратите внимание на `AutoFlush = true` у `StreamWriter` — без этого данные могут остаться в буфере и не дойти до клиента без явного вызова `Flush`.

В реальном приложении каждое подключение стоит обрабатывать в отдельной задаче, чтобы сервер мог принимать несколько клиентов одновременно:

```
while (true)
{
    TcpClient client = await server.AcceptTcpClientAsync();
    // Запускаем обработку клиента в отдельной задаче, не ожидая её
завершения
    _ = HandleClientAsync(client);
}

async Task HandleClientAsync(TcpClient client)
{
    using (client)
    await using NetworkStream ns = client.GetStream();
    using StreamReader reader = new StreamReader(ns);
    using StreamWriter writer = new StreamWriter(ns) { AutoFlush = true };

    string? message = await reader.ReadLineAsync();
    Console.WriteLine($"Получено: {message}");
    await writer.WriteLineAsync("Ответ сервера");
}
```

TcpClient — TCP-клиент

`TcpClient` используется для подключения к TCP-серверу. После установки соединения клиент предоставляет `NetworkStream`, через который происходит весь обмен данными — точно так же, как на стороне сервера.

```
using TcpClient client = new TcpClient();
await client.ConnectAsync("127.0.0.1", 8080);
Console.WriteLine("Подключено к серверу");

await using NetworkStream ns = client.GetStream();
using StreamReader reader = new StreamReader(ns);
using StreamWriter writer = new StreamWriter(ns) { AutoFlush = true };

// Отправляем сообщение
await writer.WriteLineAsync("Привет, сервер!");

// Получаем ответ
string? response = await reader.ReadLineAsync();
Console.WriteLine($"Ответ сервера: {response}");
```

NetworkStream

`NetworkStream` — это реализация `Stream` для сетевого соединения. Он наследует все возможности потоков данных, которые мы рассматривали ранее: поддерживает синхронное и асинхронное чтение и запись, совместим с декораторами (`StreamReader`, `StreamWriter`, `BinaryReader` и т.д.).

Получить `NetworkStream` можно через метод `GetStream()`:

```
await using NetworkStream ns = client.GetStream();
```

Важная особенность: `NetworkStream` не поддерживает `Seek` — данные в сетевом потоке идут последовательно и не имеют фиксированного размера, поэтому перемещение по потоку невозможно. Перед использованием всегда стоит проверять `CanRead` и `CanWrite`.

Результат работы клиента и сервера

Если запустить сервер, а затем клиент, в консоли получим:

На сервере:

```
Сервер запущен, ожидание подключений...
Клиент подключился
Получено: Привет, сервер!
```

На клиенте:

```
Подключено к серверу
Ответ сервера: Сообщение получено!
```

UdpClient — работа с UDP

`UdpClient` — высокоуровневая обёртка над UDP-сокетами. В отличие от TCP, UDP не устанавливает соединение — каждое сообщение отправляется независимо и может быть потеряно или прийти не в том порядке. Зато UDP значительно быстрее и подходит там, где важна скорость, а не гарантия доставки.

Сервер привязывается к порту и ожидает датаграммы. Метод `ReceiveAsync` возвращает `UdpReceiveResult`, который содержит и сами данные, и адрес отправителя — он нужен для того, чтобы знать, куда отправить ответ:

```
using UdpClient server = new UdpClient(8080);
Console.WriteLine("UDP-сервер запущен");

while (true)
{
    UdpReceiveResult result = await server.ReceiveAsync();
    string message = Encoding.UTF8.GetString(result.Buffer);
    Console.WriteLine($"Получено от {result.RemoteEndPoint}: {message}");

    // Отправляем ответ отправителю
    byte[] response = Encoding.UTF8.GetBytes("UDP-ответ получен");
    await server.SendAsync(response, response.Length,
result.RemoteEndPoint);
}
```

Клиент отправляет данные напрямую по адресу сервера без предварительного установления соединения:

```
using UdpClient client = new UdpClient();
IPEndPoint serverEndPoint = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
8080);

// Отправляем сообщение
byte[] data = Encoding.UTF8.GetBytes("Привет, UDP-сервер!");
await client.SendAsync(data, data.Length, serverEndPoint);

// Получаем ответ
UdpReceiveResult result = await client.ReceiveAsync();
Console.WriteLine(Encoding.UTF8.GetString(result.Buffer));
```

Управление ресурсами

Все рассмотренные классы используют сетевые ресурсы и реализуют `IDisposable`, поэтому их всегда нужно закрывать через `using`. Особенно важно корректно останавливать `TcpListener` — если этого не сделать, порт останется занятым и следующий запуск приложения завершится ошибкой:

```
var server = new TcpListener(IPAddress.Any, 8080);
server.Start();
try { //работа сервера
}
finally { server.Stop(); // освобождает порт
}
```

6. HTTP

До этого момента мы рассматривали сетевое взаимодействие на уровне соединений и передачи байтов. Однако в реальных приложениях разработчики редко работают с «сырыми» данными. Чаще всего используется **HTTP** — прикладной протокол, который определяет чёткие правила обмена сообщениями между клиентом и сервером.

HTTP (HyperText Transfer Protocol) — это протокол прикладного уровня, основанный на TCP и предназначенный для передачи запросов и ответов между клиентом и сервером, получения ресурсов по сети и взаимодействия между распределёнными сервисами.

Клиент–серверная модель HTTP

HTTP строго следует модели **клиент–сервер**: клиент инициирует запрос, сервер обрабатывает его и отправляет ответ, после чего соединение либо закрывается, либо переиспользуется.

Ключевая особенность HTTP — **stateless** (отсутствие состояния): сервер не хранит никакой информации о клиенте между запросами. Каждый запрос самодостаточен и содержит всё необходимое для его обработки. Это упрощает масштабирование серверов, но означает, что состояние (например, сессия пользователя) должно передаваться явно — через заголовки, токены или cookies.

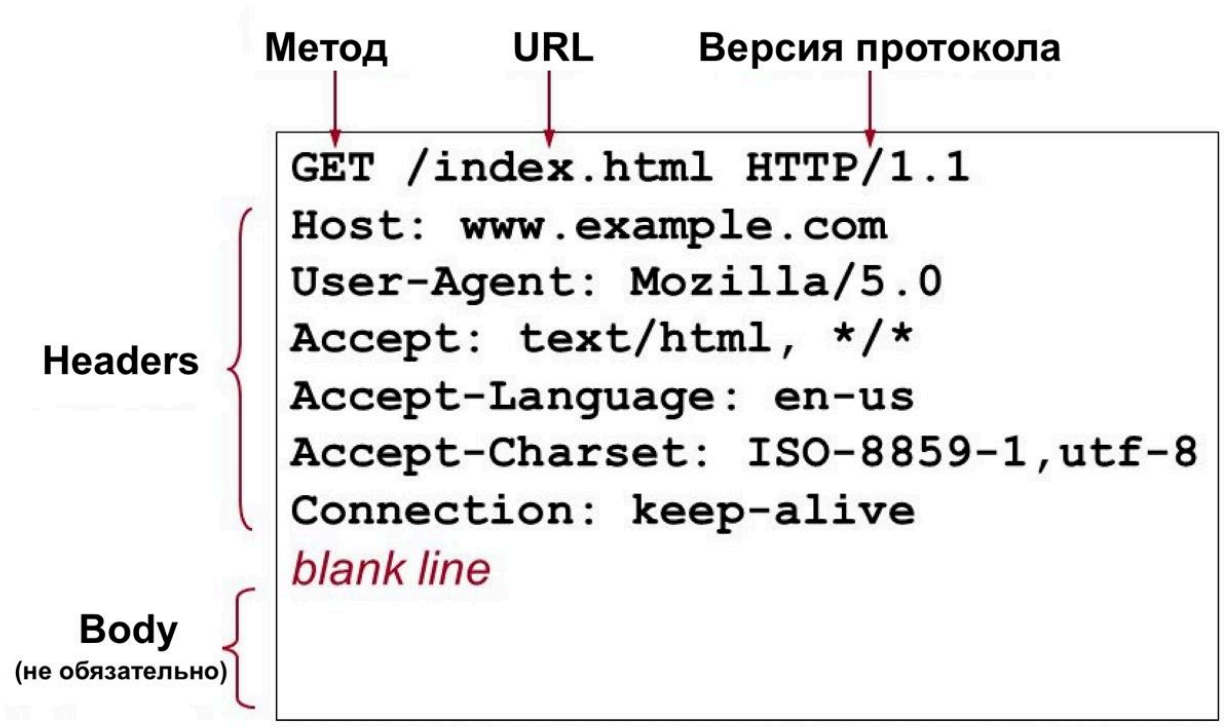
HTTP-запрос

HTTP-запрос состоит из трёх частей.

Стартовая строка содержит HTTP-метод, путь к ресурсу и версию протокола.

Заголовки (Headers) — метаданные запроса: информация о формате данных, авторизации, кодировке и т.д.

Тело запроса (Body) — передаваемые данные. Присутствует не всегда — например, GET-запросы тела не имеют.



HTTP-методы

HTTP-метод определяет **тип операции**, которую клиент хочет выполнить над ресурсом.

GET — получение данных. Не изменяет состояние сервера, тела запроса не имеет. Дополнительные параметры передаются через URL.

POST — создание ресурса или передача данных на сервер. Данные передаются в теле запроса.

PUT — полное обновление ресурса. Тело содержит новое состояние ресурса целиком.

PATCH — частичное обновление ресурса. В теле передаются только изменяемые поля.

DELETE — удаление ресурса.

Идемпотентность и безопасность методов

Методы HTTP обладают двумя важными свойствами.

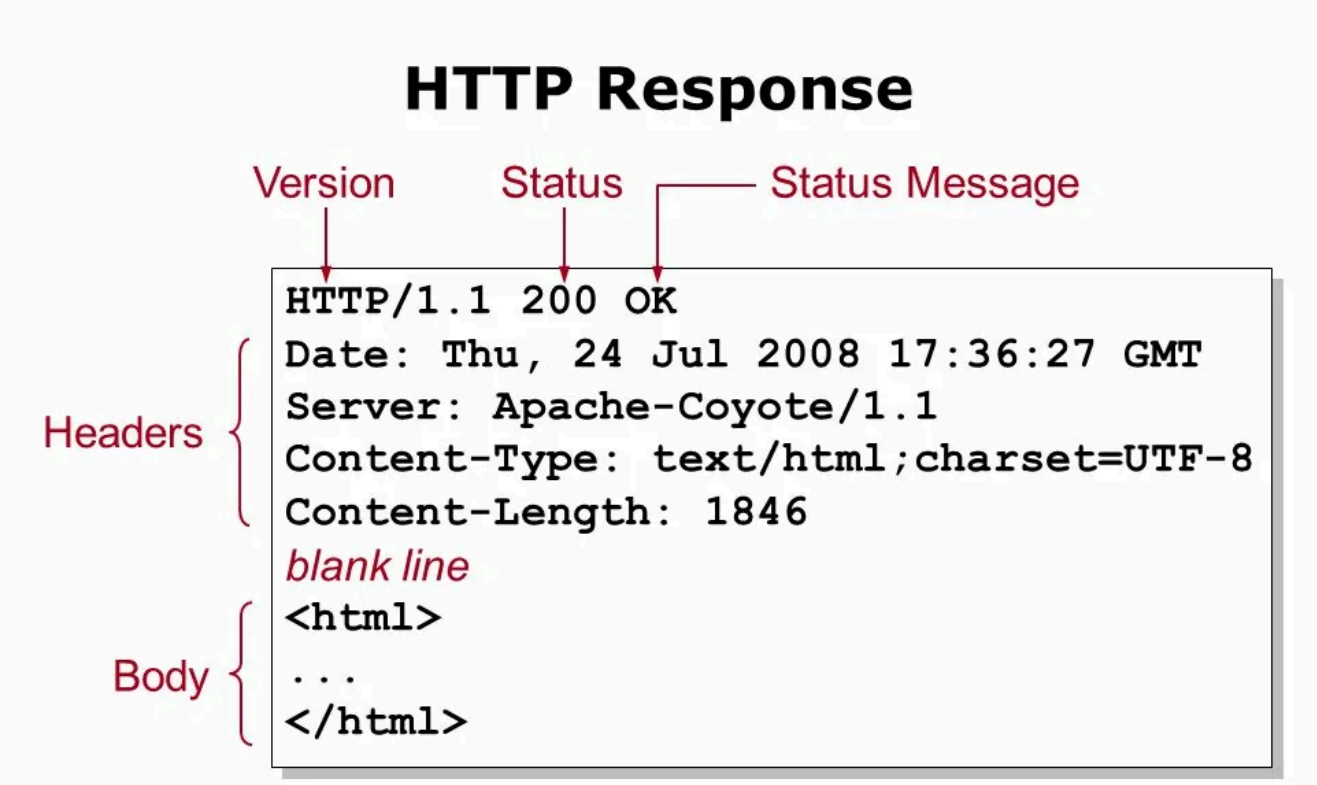
Безопасность — метод не изменяет состояние сервера. Безопасным является только GET: его можно вызывать любое количество раз без побочных эффектов.

Идемпотентность — повторный вызов с теми же параметрами даёт тот же результат, что и первый. Например, DELETE идемпотентен: удалить уже удалённый ресурс — то же самое, что удалить его один раз.

Метод	Назначение	Тело запроса	Идемпотентный	Безопасный
GET	Получение данных	Нет	Да	Да
POST	Создание ресурса	Да	Нет	Нет
PUT	Полное обновление ресурса	Да	Да	Нет
PATCH	Частичное обновление ресурса	Да	Нет	Нет
DELETE	Удаление ресурса	Нет	Да	Нет

HTTP-ответ

HTTP-ответ имеет аналогичную структуру: статусная строка (версия HTTP, код состояния и его текстовое описание), заголовки с информацией об ответе и тело с возвращаемыми данными.



Коды состояния HTTP

Код состояния сообщает результат обработки запроса. Коды сгруппированы по первой цифре:

2xx — успех. Запрос обработан корректно. Примеры: `200 OK`, `201 Created`.

3xx — перенаправление. Ресурс находится по другому адресу. Пример: `301 Moved Permanently`.

4xx — ошибка клиента. Запрос составлен неверно или не может быть выполнен. Примеры: `400 Bad Request`, `401 Unauthorized`, `404 Not Found`.

5xx — ошибка сервера. Сервер не смог обработать корректный запрос. Пример: `500 Internal Server Error`.

Знание кодов состояния важно при разработке клиентских приложений — на их основе строится логика обработки ошибок.

Заголовки HTTP

Заголовки передают дополнительную информацию вместе с запросом или ответом. Наиболее важные из них:

Content-Type — указывает формат данных в теле. Это обязательный заголовок при отправке данных на сервер, иначе сервер не сможет корректно их интерпретировать. Основные значения:

- `application/json` — JSON-данные, стандарт для REST API;
- `text/html` — HTML-страницы;
- `multipart/form-data` — загрузка файлов;
- `application/x-www-form-urlencoded` — данные формы в виде `key=value`.

Content-Length — длина тела в байтах.

7. REST API

HTTP предоставляет универсальный механизм обмена сообщениями, однако сам по себе не определяет, как проектировать серверное API. Для этого используется архитектурный стиль **REST**.

API (Application Programming Interface) — это набор правил и протоколов, который позволяет различным программам взаимодействовать друг с другом. Когда говорят о веб-API, имеют в виду интерфейс, через который клиентское приложение обращается к серверу по сети.

REST (Representational State Transfer) — архитектурный стиль, задающий набор принципов построения таких интерфейсов поверх HTTP. Важно понимать: REST — не протокол и не стандарт, а набор соглашений. Он определяет, как структурировать API, как использовать HTTP-методы и как представлять данные.

Основные принципы REST

Клиент–сервер — клиент и сервер независимы и отвечают каждый за свою часть логики. Это позволяет развивать их независимо друг от друга.

Отсутствие состояния (Stateless) — сервер не хранит состояние клиента между запросами. Каждый запрос содержит всю необходимую информацию для его обработки.

Единый интерфейс — предсказуемая структура запросов и стандартное использование HTTP-методов. Зная принципы REST, разработчик может работать с любым REST API без дополнительной документации на базовом уровне.

Кэшируемость — ответы могут кэшироваться на стороне клиента или промежуточных узлов, что повышает производительность.

Ресурсы и эндпоинты

Ключевой абстракцией в REST является **ресурс** — любая сущность, которую приложение предоставляет внешнему миру.

Каждому ресурсу назначается URI — адрес, по которому к нему можно обратиться. Для каждого ресурса определяется список допустимых операций. Пара из HTTP-метода и URI называется **эндпоинтом (endpoint)**:

- `GET /api/users` — получить список пользователей;
- `POST /api/users` — создать пользователя;
- `GET /api/users/5` — получить пользователя с ID=5;
- `DELETE /api/users/5` — удалить пользователя с ID=5.

CRUD и HTTP-методы

Для каждого ресурса обычно определяют четыре типа операций: **создание (Create), чтение (Read), обновление (Update), удаление (Delete)** — сокращённо **CRUD**. Каждая операция соответствует одному или нескольким HTTP-методам:

Операция	HTTP-метод	Пример URL
Create	POST	`/users`
Read	GET	`/users/42`
Update	PUT / PATCH	`/users/42`
Delete	DELETE	`/users/42`

Если мы хотим спроектировать RESTful API, нужно: определить ресурсы системы, назначить им URI, реализовать CRUD-операции и определить формат передаваемых данных — структуру JSON в теле запроса, параметры URL и коды ответов.

Форматы данных: JSON

REST не привязан к конкретному формату данных, но на практике стандартом является **JSON**. Подробно структура JSON и работа с ним в C# рассматриваются в следующем разделе. Здесь важно понять принцип: сервер возвращает данные в виде JSON и принимает JSON в теле запроса, что делает взаимодействие симметричным и предсказуемым.

Версионирование API

Со временем API изменяется: добавляются новые поля, меняется структура ответов. Чтобы не сломать существующих клиентов, используется **версионирование**. Наиболее распространённый подход — версия в URI:

```
GET /api/v1/users
GET /api/v2/users
```

Это позволяет развивать API и одновременно поддерживать несколько версий для разных клиентов.

8. JSON

JSON (JavaScript Object Notation) — это легкий текстовый формат обмена данными, который легко читается человеком и быстро обрабатывается машинами. Несмотря на то что он произошел от JavaScript, формат поддерживается практически всеми современными языками программирования.

JSON строится из двух базовых конструкций — **объектов** и **массивов**.

Объект — набор пар «ключ: значение» в фигурных скобках. Ключи всегда строки, значения могут быть разных типов:

```
{
  "id": 1,
  "name": "Иван Иванов",
  "age": 25,
  "isActive": true,
  "score": 98.5,
  "nickname": null
}
```

Поддерживаемые типы значений: `string` (строка в кавычках), `number` (целое или дробное), `boolean` (`true` / `false`), `null`, вложенный объект и массив.

Массив — упорядоченный список значений в квадратных скобках:

```
["яблоко", "банан", "вишня"]
```

Объекты и массивы могут содержать друг друга, что позволяет описывать сложные вложенные структуры:

```
{
  "id": 1,
  "name": "Иван Иванов",
  "address": {
    "city": "Москва",
    "street": "Ленина",
    "building": 42
  },
  "hobbies": ["чтение", "программирование", "шахматы"],
  "orders": [
    { "orderId": 101, "total": 1500.0 },
    { "orderId": 102, "total": 320.5 }
  ]
}
```

Сериализация и десериализация

Чтобы передать объект C# по сети, его нужно преобразовать в текст — это называется **сериализацией**. Обратный процесс — восстановление объекта из текста — называется **десериализацией**.

System.Text.Json

В .NET основным инструментом для работы с JSON является пространство имён `System.Text.Json`, встроенное в платформу. Центральный класс — `JsonSerializer`.

Предположим, у нас есть класс:

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Сериализация — преобразование объекта в JSON-строку:

```
var user = new User { Id = 1, Name = "Иван Иванов", Email =
"ivan@example.com" };
string json = JsonSerializer.Serialize(user);
Console.WriteLine(json);
// {"Id":1,"Name":"Иван Иванов","Email":"ivan@example.com"}
```

Десериализация — восстановление объекта из JSON-строки:

```
string json = "{\"Id\":1,\"Name\":\"Иван
Иванов\",\"Email\":\"ivan@example.com\"}";
User? user = JsonSerializer.Deserialize<User>(json);
Console.WriteLine(user.Name); // Иван Иванов
```

Сериализация и десериализация работают также со списками:

```
var users = new List<User>
{
    new User { Id = 1, Name = "Иван" },
    new User { Id = 2, Name = "Мария" }
};

string json = JsonSerializer.Serialize(users);
List<User>? result = JsonSerializer.Deserialize<List<User>>(json);
```

Настройка через JsonSerializerOptions

Поведение сериализатора можно настраивать через `JsonSerializerOptions`. Наиболее частые сценарии:

camelCase — по умолчанию `JsonSerializer` сохраняет имена свойств как есть (`Name`, `Email`). REST API чаще всего ожидают `camelCase` (`name`, `email`):

```
var options = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
};

string json = JsonSerializer.Serialize(user, options);
// {"id":1,"name":"Иван Иванов","email":"ivan@example.com"}
```

Игнорирование null — по умолчанию null-поля включаются в JSON. Их можно исключить:

```
var options = new JsonSerializerOptions
{
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};
```

Форматирование — для удобства чтения можно включить отступы:

```
var options = new JsonSerializerOptions
{
    WriteIndented = true
};
```

Нечувствительность к регистру при десериализации — полезно, если сервер возвращает `name`, а свойство называется `Name`:

```
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true
};
```

Атрибуты

Атрибуты позволяют управлять сериализацией на уровне отдельных свойств.

`[JsonPropertyName]` — задаёт имя свойства в JSON независимо от имени в C#:

```
public class User
{
    [JsonPropertyName("user_id")]
    public int Id { get; set; }

    [JsonPropertyName("full_name")]
    public string Name { get; set; }
}
```

```
{ "user_id": 1, "full_name": "Иван Иванов" }
```

`[JsonIgnore]` — исключает свойство из сериализации:

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }

    [JsonIgnore]
    public string PasswordHash { get; set; } // не попадёт в JSON
}
```

9. HttpClient и HttpListener

После изучения принципов HTTP, REST API и работы с JSON перейдём к практической стороне — как отправлять HTTP-запросы и принимать их в C#. Для этого в .NET предусмотрены два класса: `HttpClient` на стороне клиента и `HttpListener` на стороне сервера.

HttpClient

`HttpClient` — основной инструмент для выполнения HTTP-запросов в .NET. Он поддерживает все HTTP-методы, работает асинхронно и хорошо интегрируется с сериализацией JSON.

```
using var client = new HttpClient();
var response = await client.GetAsync(url);
```

Правильное создание и переиспользование

`HttpClient` управляет пулом соединений внутри себя. Если создавать новый экземпляр на каждый запрос и сразу его уничтожать, соединения не успевают закрываться на уровне ОС, что приводит к исчерпанию сокетов под нагрузкой.

Правильный подход — использовать один долгоживущий экземпляр:

Также можно задать базовый адрес, чтобы не повторять его в каждом запросе:

```
_client.BaseAddress = new Uri("https://api.example.com/");
```

GET-запросы

Простейший GET-запрос — получить данные и прочитать ответ как строку:

```
HttpResponseMessage response = await
_client.GetAsync("https://api.example.com/users");
response.EnsureSuccessStatusCode();

string content = await response.Content.ReadAsStringAsync();
Console.WriteLine(content);
```

Если нужно сразу получить десериализованный объект, удобнее использовать `GetFromJsonAsync` — он читает тело ответа и десериализует его за один вызов:

```
User? user = await _client.GetFromJsonAsync<User>
("https://api.example.com/users/1");
```

POST-запросы с JSON

Для отправки объекта в теле запроса проще всего использовать `PostAsJsonAsync` — он автоматически сериализует объект и выставляет заголовок `Content-Type: application/json`:

```
var newUser = new User { Name = "Мария", Email = "maria@example.com" };

HttpResponseMessage response = await _client.PostAsJsonAsync(
    "https://api.example.com/users",
    newUser
);
```

Если нужно передать кастомные настройки сериализации, можно сериализовать вручную:

```
var options = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
};

string json = JsonSerializer.Serialize(newUser, options);
var content = new StringContent(json, Encoding.UTF8, "application/json");

HttpResponseMessage response = await _client.PostAsync(
    "https://api.example.com/users",
    content
);
```

Аналогично работают `PutAsJsonAsync` и `PatchAsJsonAsync` для обновления ресурсов, и `DeleteAsync` для удаления.

Работа с HTTP-ответом

Метод возвращает `HttpResponseMessage`, который содержит статус-код, заголовки и тело ответа:

```
HttpResponseMessage response = await _client.GetAsync(url);

Console.WriteLine(response.StatusCode);        // OK, NotFound, ...
Console.WriteLine(response.IsSuccessStatusCode); // true / false

// Читаем тело как строку
string text = await response.Content.ReadAsStringAsync();
```

```
// Или сразу десериализуем
User? user = await response.Content.ReadFromJsonAsync<User>();
```

`EnsureSuccessStatusCode()` выбрасывает `HttpRequestException`, если код ответа не из диапазона 2xx. Это удобный способ быстро прервать выполнение при ошибке, однако в сложной логике лучше проверять статус-код явно:

```
if (response.StatusCode == HttpStatusCode.NotFound)
{
    Console.WriteLine("Пользователь не найден");
    return null;
}
```

Заголовки

Заголовки можно задавать глобально для всех запросов клиента или для конкретного запроса.

Глобальные заголовки — например, токен авторизации:

```
_client.DefaultRequestHeaders.Add("Authorization", "Bearer token123");
_client.DefaultRequestHeaders.UserAgent.ParseAdd("MyApp/1.0");
```

Для заголовков конкретного запроса используется `HttpRequestMessage`:

```
var request = new HttpRequestMessage(HttpMethod.Get,
"https://api.example.com/users");
request.Headers.Add("X-Custom-Header", "value");

HttpResponseMessage response = await _client.SendAsync(request);
```

Читать заголовки ответа можно через `response.Headers`:

```
if (response.Headers.TryGetValues("X-RateLimit-Remaining", out var values))
    Console.WriteLine($"Осталось запросов: {values.First()}");
```

Таймауты и CancellationToken

Таймаут задаётся через свойство `Timeout` — если сервер не ответил за отведённое время, выбрасывается `TaskCanceledException`:

```
_client.Timeout = TimeSpan.FromSeconds(10);
```

Для более гибкого управления используется `CancellationToken` — он позволяет отменить запрос по внешнему сигналу, например при закрытии приложения или по действию пользователя:

```
var cts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
try
{
    HttpResponseMessage response = await _client.GetAsync(url, cts.Token);
}
catch (TaskCanceledException)
{
    Console.WriteLine("Запрос отменён или превышено время ожидания");
}
```

Обработка ошибок

Сетевые операции могут завершаться неудачей по множеству причин — разрыв соединения, недоступность сервера, таймаут, некорректный ответ. Всегда оборачивайте запросы в `try-catch`:

```
try
{
    HttpResponseMessage response = await _client.GetAsync(url);
    response.EnsureSuccessStatusCode();
    var user = await response.Content.ReadFromJsonAsync<User>();
}
catch (HttpRequestException ex) // Сетевая ошибка или код ответа не 2xx
{
    Console.WriteLine($"HTTP-ошибка: {ex.StatusCode} - {ex.Message}");
}
catch (TaskCanceledException) // Таймаут или отмена через CancellationToken
{
    Console.WriteLine("Запрос отменён или превышено время ожидания");
}
catch (JsonException ex) // Сервер вернул некорректный JSON
{
    Console.WriteLine($"Ошибка десериализации: {ex.Message}");
}
```

Пример: клиент для REST API

Рассмотрим пример полноценного клиента для работы с API сервиса по прогнозу погоды:

```
public class WeatherClient
{
    private readonly HttpClient _client;
    private const string ApiKey = "ваш_ключ";
    public WeatherClient()
    {
        _client = new HttpClient
        {
            BaseAddress = new Uri("https://api.weatherapi.com/v1/"),
            Timeout = TimeSpan.FromSeconds(10)
        };
    }
    public async Task<WeatherData?> GetWeatherAsync(string city,
CancellationToken token = default)
    {
        try
        {
            string url = $"weather?key={ApiKey}&q=
{Uri.EscapeDataString(city)}";
            HttpResponseMessage response = await _client.GetAsync(url,
token);
            response.EnsureSuccessStatusCode();
            return await response.Content.ReadFromJsonAsync<WeatherData>
(cancellationToken: token);
        }
        catch (HttpRequestException ex)
        {
            Console.WriteLine($"Ошибка запроса: {ex.Message}");
            return null;
        }
    }
}
```

HttpListener

`HttpListener` позволяет запустить простой HTTP-сервер прямо в консольном приложении. Он работает на уровне операционной системы и слушает указанные адреса и порты.

Типичный жизненный цикл: создать `HttpListener`, добавить префиксы (адреса для прослушивания), запустить, в цикле принимать запросы и отвечать на них.

```
var listener = new HttpListener();
listener.Prefixes.Add("http://localhost:8080/");
listener.Start();
Console.WriteLine("Сервер запущен: http://localhost:8080/");
while (true)
{
    // Ожидаем входящий запрос
    HttpListenerContext context = await listener.GetContextAsync();
    HttpListenerRequest request = context.Request;
    HttpListenerResponse response = context.Response;
    Console.WriteLine($"{request.HttpMethod} {request.Url}");
    // Формируем ответ
    string responseText = "Привет от HttpListener!";
    byte[] buffer = Encoding.UTF8.GetBytes(responseText);
    response.ContentType = "text/plain; charset=utf-8";
    response.ContentLength64 = buffer.Length;
    response.StatusCode = 200;
    await response.OutputStream.WriteAsync(buffer);
    response.Close(); // обязательно закрываем ответ
}
```

Пример: простой REST-сервер на HttpListener

Рассмотрим сервер, который умеет обрабатывать GET и POST запросы, читать тело запроса и возвращать JSON:

```
var listener = new HttpListener();
listener.Prefixes.Add("http://localhost:8080/");
listener.Start();

while (true)
{
    HttpListenerContext context = await listener.GetContextAsync();
    _ = HandleRequestAsync(context); // обрабатываем каждый запрос в
отдельной задаче
}
```

```
async Task HandleRequestAsync(HttpListenerContext context)
{
    HttpListenerRequest request = context.Request;
    HttpListenerResponse response = context.Response;

    try
    {
        if (request.HttpMethod == "GET" && request.Url?.AbsolutePath ==
"/users")
        {
            // Возвращаем список пользователей
            var users = new[]
            {
                new { Id = 1, Name = "Иван" },
                new { Id = 2, Name = "Мария" }
            };

            string json = JsonSerializer.Serialize(users);
            byte[] buffer = Encoding.UTF8.GetBytes(json);

            response.ContentType = "application/json; charset=utf-8";
            response.ContentLength64 = buffer.Length;
            response.StatusCode = 200;
            await response.OutputStream.WriteAsync(buffer);
        }
        else if (request.HttpMethod == "POST" && request.Url?.AbsolutePath
== "/users")
        {
            // Читаем тело запроса
            using var reader = new StreamReader(request.InputStream,
Encoding.UTF8);
            string body = await reader.ReadToEndAsync();
            var newUser = JsonSerializer.Deserialize<User>(body);

            Console.WriteLine($"Создан пользователь: {newUser?.Name}");

            response.StatusCode = 201;
        }
        else
        {
            response.StatusCode = 404;
        }
    }
    finally
    {
        response.Close();
    }
}
```

Практическое задание

Исходная ситуация

В рамках решения уже реализовано локальное файловое хранилище (`FileManager`), работающее через `Stream` и поддерживающее шифрование данных.

Теперь требуется добавить сетевое взаимодействие и синхронизацию данных через HTTP.

Что нужно сделать

Создать клиент-серверную архитектуру внутри текущего решения:

1. Добавить новый проект — HTTP-сервер.
2. Реализовать сервер на основе `HttpListener`.
3. В основном проекте реализовать API-клиент на `HttpClient`.
4. Создать класс `ApiDataStorage`, реализующий `IDataStorage`.
5. Добавить команду `sync` для синхронизации с сервером.
6. Обеспечить передачу **зашифрованных данных**.

Часть 1. Серверный проект

Создать новый проект в том же Solution:

```
TodoList.Server
```

Требования к серверу

1. Сервер должен использовать `HttpListener`.
2. Сервер запускается как отдельное консольное приложение.
3. Сервер должен слушать адрес:

```
http://localhost:5000/
```

4. Сервер должен работать в бесконечном цикле обработки запросов.
5. Обработка запросов должна быть асинхронной.

Поддерживаемые endpoints

1. `POST /profiles`
   - Принимает зашифрованные данные профилей.
   - Сохраняет их в файл.
   - Не расшифровывает данные.

- Не работает с моделями `Profile`.

2. `GET /profiles`
   - Возвращает сохранённые зашифрованные данные профилей.
3. `POST /todos/{userId}`
   - Принимает зашифрованные данные задач пользователя.
   - Сохраняет их в файл:

   ```
   server_todos_{userId}.dat
   ```

4. `GET /todos/{userId}`
   - Возвращает зашифрованные данные задач пользователя.

Важно

Сервер:

- не знает о `Profile`
- не знает о `TodoItem`
- не расшифровывает данные
- не использует `IDataStorage`
- работает только со строками/байтами

Часть 2. Клиентская часть

В основном проекте необходимо создать класс:

```
public class ApiDataStorage : IDataStorage
```

Требования к реализации

1. Использовать `HttpClient`.
2. Методы класса должны полностью соответствовать интерфейсу:

```
public interface IDataStorage
{
    void SaveProfiles(IEnumerable<Profile> profiles);
    IEnumerable<Profile> LoadProfiles();

    void SaveTodos(Guid userId, IEnumerable<TodoItem> todos);
    IEnumerable<TodoItem> LoadTodos(Guid userId);
}
```

3. Перед отправкой данных:
   - сериализовать в JSON
   - зашифровать (AES + CryptoStream)

4. После получения данных:
    - расшифровать
    - десериализовать

Поведение методов

SaveProfiles

1. Сериализация
2. Шифрование
3. POST `/profiles`

LoadProfiles

1. GET `/profiles`
2. Расшифровка
3. Десериализация

Аналогично для задач пользователя.

Часть 3. Команда `sync`

Добавить новую команду:

```
sync
```

С двумя флагами `--pull` и `--push`

Логика работы

1. Проверить доступность сервера.
2. Если сервер не запущен — вывести:

```
Ошибка: сервер недоступен.
```

3. Если сервер доступен:
    1. Если установлен флаг `--pull`
        - получить данные с сервера
        - обновить локальное состояние
    2. Если установлен флаг `--push`
        - отправить локальные данные на сервер
        - перезаписать данные на сервере